What Is the Boat Trying to Do?

Open loop first, then PID, LQR, and pole placement

J.C. Vaught October 11, 2026

1 The objective. Return to a line and follow it

The previous report chose a moving-line test to compare controllers. This revision begins with the physical task. The boat should recover an east-going route from an initial position and heading error, then keep traveling close to that route at a forward speed of 1 m/s. This is a selected test objective, not a complete competition mission.

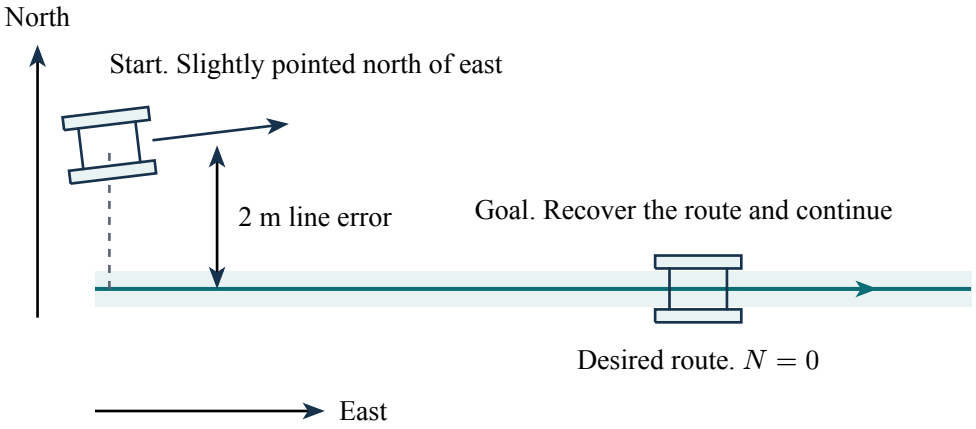


Figure 1: Start north of the route. Recover it, then continue east.

- Where should it go?
  - Follow the line $N_d = 0$ m, with east to the right and north upward.
    - The hull origin starts at $(E, N) = (0, 2)$ m. Its bow points 0.12 rad, or 6.9°, north of east. It begins at 0.8 m/s forward speed.
- How should it move?
  - Bring its north position toward zero while bringing forward speed u toward 1 m/s.
    - Forward speed is along the boat's own axis. East ground speed is not a separately controlled reference, and there is no required destination or arrival time.
- What counts as recovery?
  - After the transient, keep line distance within 0.25 m and forward speed within 0.1 m/s of the reference.
    - A settled run stays inside both bounds through the end of the 40 s test. The report also gives continuous error scores so that a late gust does not hide otherwise good tracking.

> Read the report in order. First identify the target route, then watch fixed equal thrust, then add feedback. The mathematical controller design is retained in the appendix after the visual comparison.

2 First watch the boat without feedback

The simplest starting point is a constant motor command. Choose the nominal thrust that balances forward drag at 1 m/s, apply it to both motors, and leave it unchanged. The boat then responds to its initial heading, hydrodynamics, motor differences, and wind; no sensor reading changes the command.

- The open-loop command is approximately 1589.9 microseconds to each motor.
  - The nominal installed M200 map converts this to 7 N per motor, or 14 N total.
    - This balances the model's nominal forward drag at 1 m/s. It does not request a north position, a heading correction, or a turn toward the line.
- The computer retransmits that fixed command at 50 Hz.
  - Sensors still operate and command delay, motor response, and other enabled simulator effects still apply.
    - The open-loop policy ignores every observation. It is a deliberate fixed-trim baseline; an open-loop program could also contain predetermined turns.

2.1 Why a perfect start can be misleading

Begin with calm water, matched motors, and no lost commands. This illustration retains the nonlinear hull and M200 models. Change only the initial state to see why a boat that looks correct in one open-loop run may fail the actual recovery task.

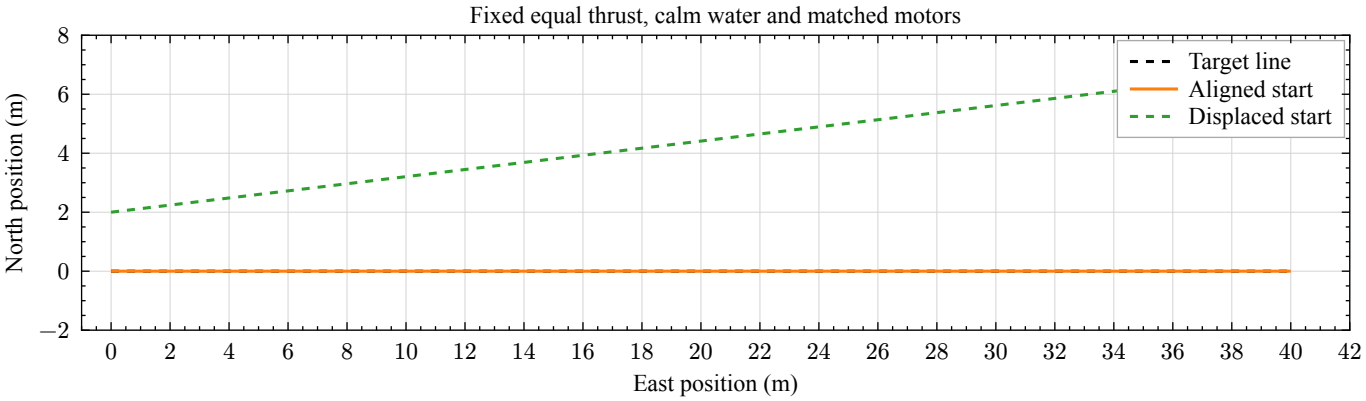


Figure 2: The same fixed command succeeds from an aligned start and misses the displaced-start objective.

- Aligned start. The boat begins on the line, points east, and already travels at 1 m/s.
  - It reaches approximately 40 m east while staying on the line over 40 s.
    - No correction is needed, so this case does not demonstrate that the command can correct an error.
- Displaced start. The boat begins 2 m north, points 6.9° north of east, and travels initially at 0.8 m/s.
  - Forward speed rises toward 1 m/s, but the heading stays wrong and north position reaches 6.76 m.
    - The boat has regulated its speed through passive drag balance while failing the route objective. Equal fixed thrust supplies no instruction to return to the line.

> Speed and route following are different goals. A boat can move at the desired speed while traveling along the wrong path.

3 Now add the full boat model and feedback

The preceding illustration isolated the initial-condition problem. Return next to the complete model, including 5% port/starboard motor mismatch, added inertia, timing errors, measurement imperfections, and environmental forcing. The following comparison keeps the same initial condition and wind seed for open loop and all three feedback controllers.

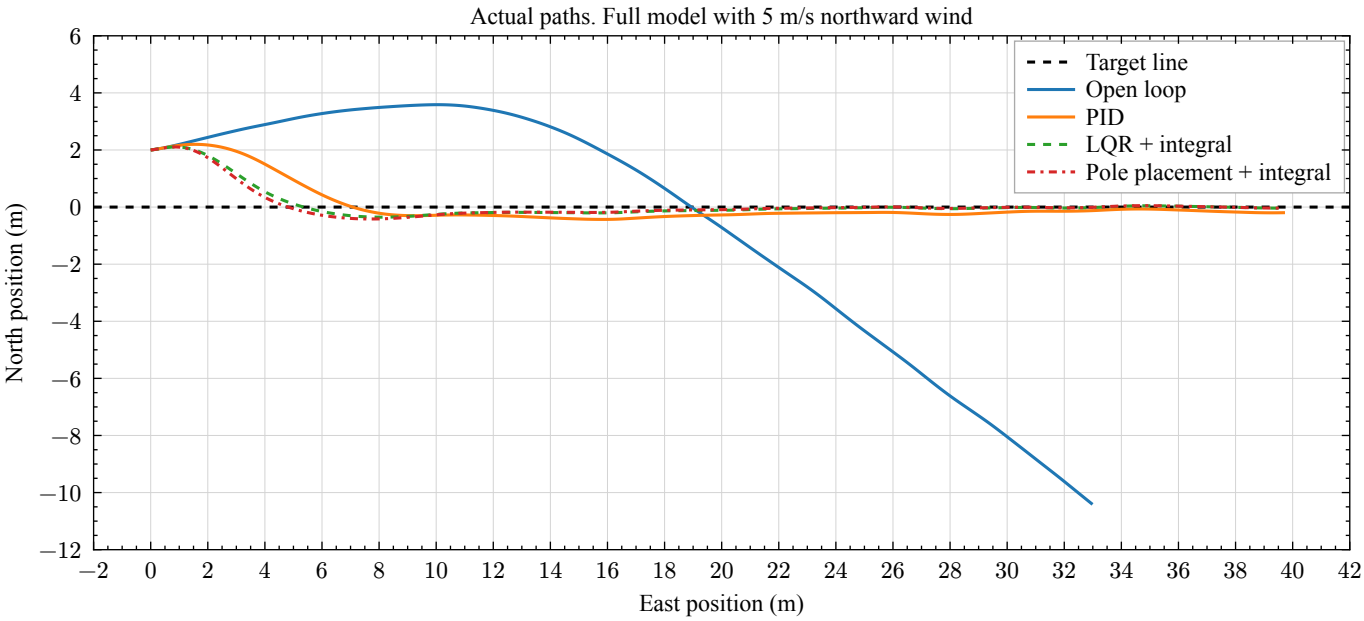


Figure 3: Actual paths in 5 m/s northward wind, seed 792. East and north use the same metre scale.

- Read the map as a top-down view.
  - East is right and north is up. The dashed horizontal line is the desired route; every boat begins at $(0, 2)$ m.
    - The fixed-command boat finishes far south of the line. The feedback boats turn toward the line and then continue close to it. Their paths overlap because their errors are small on this map scale.
- All feedback controllers receive the same type of delivered GNSS and IMU data.
  - PID uses speed, heading, and cross-track loops. LQR and pole placement use estimated state and integral feedback.
    - Each changes port and starboard commands when the estimated boat state differs from the objective. The detailed gains and equations follow in the appendix.
- Equal PWM does not ensure equal actual thrust in the full model.
  - The model multiplies nominal port thrust by 0.95 and starboard thrust by 1.05.
    - In calm water this bias bends the fixed-command path north. At 40 s, its north position is 29.12 m on average across the three seeds.

3.1 Why a northward wind can produce a southward path

The wind acts on sway and yaw as well as on forward motion. In this model, positive sway also changes yaw through the added-inertia coupling term $(m_x - m_y)uv$. Because $m_y > m_x$, positive sway contributes a clockwise yaw moment. That contribution competes with the direct wind yaw moment and motor bias.

- The resulting open-loop bow turns south of east in the saved wind runs.
  - Its forward velocity then has a southward world component, which can outweigh its northward body-frame sway velocity.
    - This is why the boat's final displacement is not obtained by simply adding a northward drift to a straight east-going track.
- The controllers correct the resulting route error rather than assuming which way the wind will move the boat.
  - These curves are predictions of the configured model, including its assumed added inertia and wind coefficients.
    - Wind direction alone is insufficient to predict the boat's path without considering heading and coupled dynamics.

4 Check the two parts of the objective separately

The map shows where the boat goes. The plots below show when the objective is met. Positive line error means north of the route; negative error means south. Speed error is measured along the boat's own forward axis.

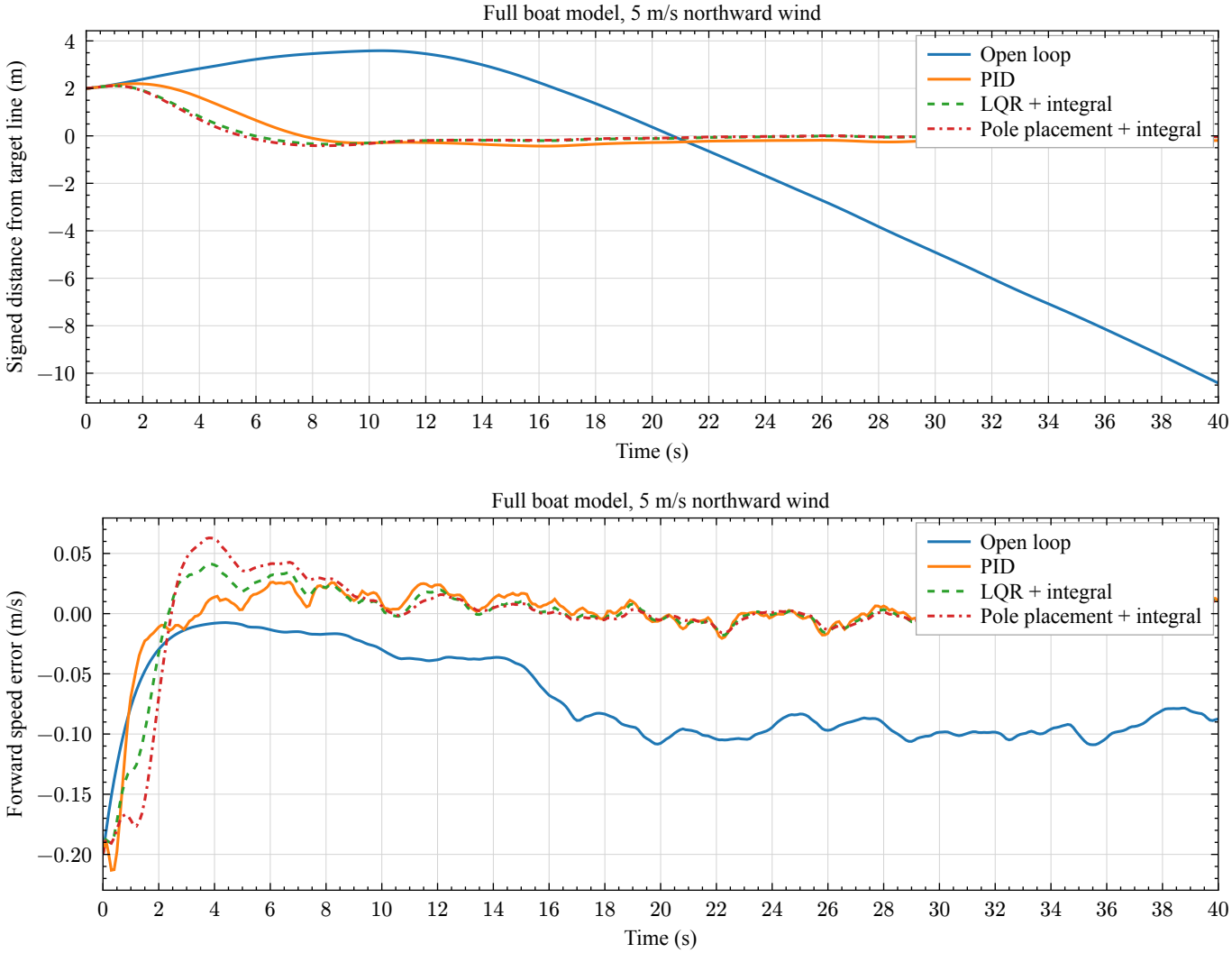


Figure 4: Route error and forward-speed error in the same 5 m/s run.

- The upper plot answers whether the boat returns to the route.
  - Zero is the desired line. The initial value is 2 m for every policy.
    - Feedback brings the error close to zero. Fixed equal thrust cannot correct the changing path.
- The lower plot answers whether forward speed approaches 1 m/s.
  - Zero means the speed target is met; this does not imply that the boat is on the desired route.
    - A small speed error can coexist with a large line error, as the open-loop example demonstrates.

The table scores the final ten seconds, after the initial recovery. Values are root-mean-square (RMS) line distances averaged across seeds 792, 793, and 794. A value of 0.05 m means a 5 cm RMS route error over that window, rather than a constant 5 cm offset.

Environment	Open loop (m)	PID (m)	LQR (m)	Pole placement (m)
Calm	24.331	0.163	0.011	0.004
5 m/s wind	8.057	0.181	0.034	0.034
10 m/s wind	6.127	0.229	0.056	0.053

LQR and pole placement remain close in this moving-line benchmark. PID has a slower recovery and greater residual line error with its saved tuning. The open-loop comparison establishes why feedback is useful before comparing which feedback design performs best.

5 What the controller changes and what the sensors measure

The error plots explain the outcome. The motor commands explain the action that produces it. Open loop sends the same PWM to both motors; feedback changes their sum to regulate forward motion and their difference to steer.

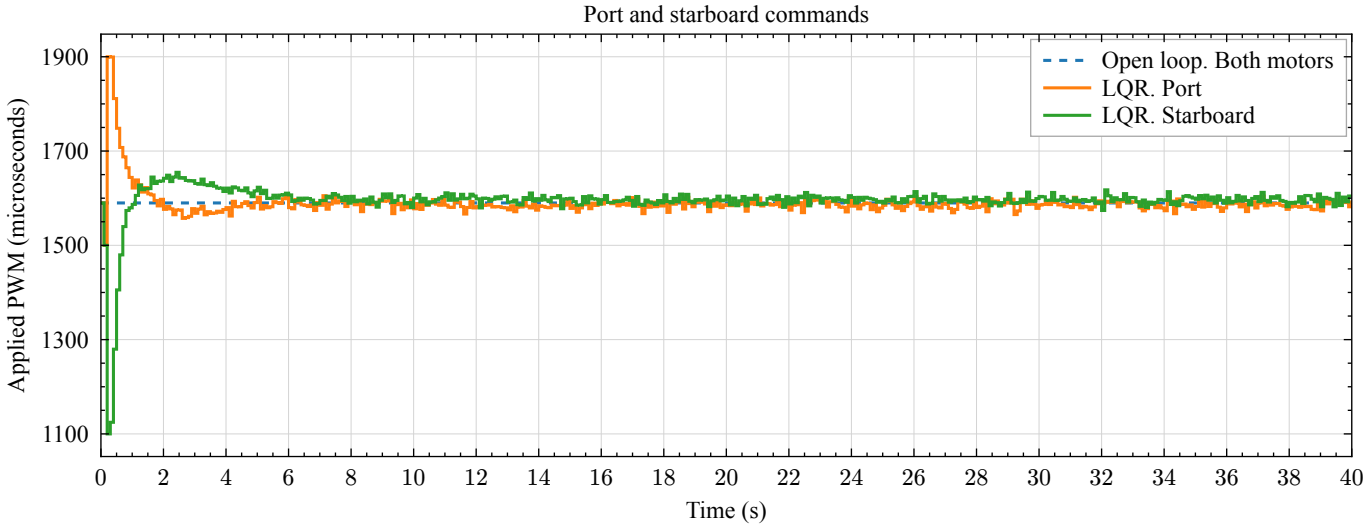


Figure 5: Fixed open-loop commands and changing LQR commands in 5 m/s wind, seed 792.

- Increasing both motor commands increases nominal forward thrust.
  - Increasing starboard thrust relative to port gives a counterclockwise yaw moment.
    - A corrective turn may require one motor to reduce thrust or reverse briefly. Actual force still follows the nonlinear lookup, lag, and slew limit.
- The line coordinate and desired speed come from the reference, not from a sensor.
  - GNSS measures antenna position, world velocity, and heading. The IMU measures yaw rate; the shared estimator corrects the antenna offset and extrapolates delayed data.
    - The controller computes line and speed errors from that estimate. Stereo camera observations remain simulated but are unused in this route-following benchmark.

5.1 Bow direction and travel direction are different

The following snapshot uses the LQR boat at 20 s in 10 m/s wind. The bow points into the wind while motion remains nearly along the east-going route. The hull drawing is enlarged for clarity; arrow directions come from the simulated state.

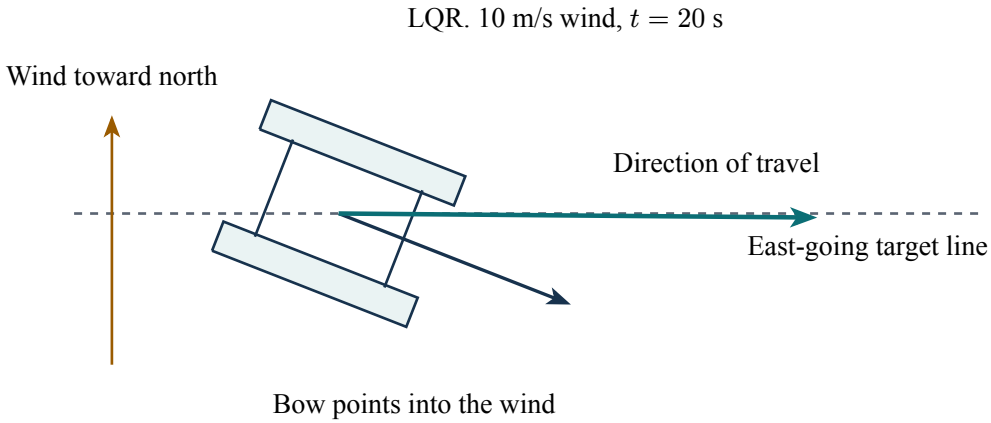


Figure 6: A crab heading can hold the route while the boat moves east.

- Heading ψ says where the bow points.
  - Direction of travel follows world velocity, measured by GNSS.
    - With forward velocity u and sideways velocity v, the world components are $V_E = u\cos\psi - v\sin\psi$ and $V_N = u\sin\psi + v\cos\psi$.
- A successful wind-compensating boat does not always point exactly east.
  - The controller can hold a southward heading so that forward motion cancels the northward sideways component.
    - For this task, score line error and forward speed rather than treating a nonzero heading as failure.

6 Use the flags to explain the difference

Once the path and objective are clear, the simulator flags become experiments rather than a list of implementation details. Every stage below uses the same initial condition, reference, fixed open-loop command, and frozen feedback gains. Command sampling is enabled in every stage so that all policies share a 50 Hz decision clock.

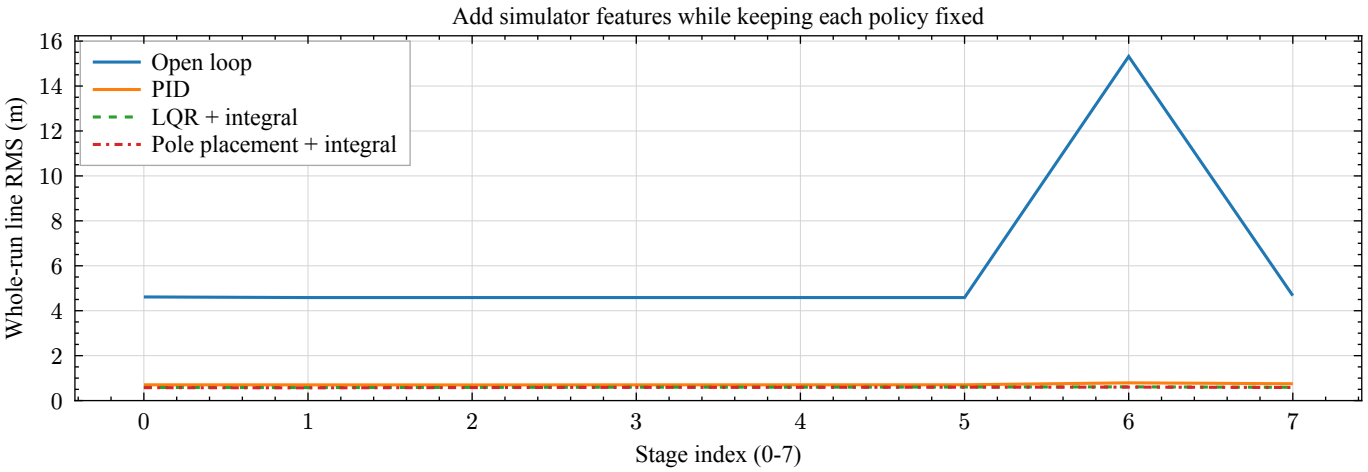


Figure 7: Open-loop and feedback line error as model features are added.

Stage	Preset	Added behavior
0	`00_linear`	Local hull and tangent thrust; perfect hull feedback.
1	`01_hull`	Nonlinear hull rotation, coupling, and drag.
2	`02_m200`	Nonlinear signed thrust, saturation, deadband, reverse asymmetry.
3	`03_motor`	Motor lag and force slew limit.
4	`04_timing`	Command delay.
5	`05_sensors`	Measured feedback with noise, bias, delay, and antenna offset.
6	`06_disturbances`	Wind, gusts, apparent wind, wind moment, waves.
7	`07_uncertainty`	Added inertia, motor mismatch, jitter, command loss, watchdog, sensor loss.

- The line error already exists at the simplest stage.
  - The open-loop boat starts displaced and never receives a correction to that objective.
    - Added features change its path; they do not supply missing feedback.
- The three feedback methods retain much smaller route errors across the ladder.
  - The primary study uses the same inverse M200 map for all policies that request force.
    - Early tangent-thrust stages therefore include a mismatch with that physical map compensation. No controller gain is retuned between stages.
- The curve need not rise monotonically as features are added.
  - Added damping, inertia, and yaw coupling can change both the disturbance response and the direction of travel.
    - A more detailed simulation is a different model, not automatically a more accurate boat prediction.

6.1 Reproduce the comparison

The 171 saved feedback runs remain unchanged. This revision adds 39 fixed-command runs, including the nominal illustrations, matched full-model environments, and the same eight-stage ladder. Report maps use seed 792; summary values include all three seeds.

```
uv sync
uv run python -m blueboat_sim.open_loop_study
typst compile controller_report.typ controller_report.pdf
```

- `data/open_loop.json` retains commands, configurations, scores, paths, and source hashes.
  - Full truth CSVs and command records are under `build/open-loop/`.
    - Existing feedback data remain unchanged; regenerate them with `uv run python -m blueboat_sim.controller_study`.
- All 24 project tests pass, including fixed-command independence from observations, nominal straight-line trim, and failure to correct an initial heading/line error.
  - The preceding controller study also audited its 171 runs and checked integration-step refinement.

A Implementation and mathematical reference

The preceding maps establish the objective and observed behavior. This appendix gives the sensor processing, limits, model, and gains used to produce those results.

A.1 Causal state estimate

The controllers need body-frame velocities and a current hull position. GNSS measures a displaced antenna and arrives late, so the shared estimator first corrects its lever arm and then extrapolates the corrected measurement to the decision time. It retains the most recent valid record through random dropouts.

The antenna offset is $\ell = -0.5$ m along the forward axis. At measurement heading ψ_g and yaw rate estimate $\hat{r}$, the hull origin and world velocity satisfy the following expressions.

$$\widehat{E_g} = E_a - \ell \cos\psi_g, \quad \widehat{N_g} = N_a - \ell \sin\psi_g. \tag{1}$$

$$\widehat{V_E} = V_{E,a} + \ell \hat{r} \sin\psi_g. \tag{2}$$

$$\widehat{V_N} = V_{N,a} - \ell \hat{r} \cos\psi_g. \tag{3}$$

For measurement age $\Delta = t - t_g$, constant world velocity and yaw rate give the decision-time estimate.

$$\hat{E} = \widehat{E_g} + \Delta \widehat{V_E}, \quad \hat{N} = \widehat{N_g} + \Delta \widehat{V_N}. \tag{4}$$

$$\hat{\psi} = \mathrm{wrap}\,\left(\psi_g + \Delta \hat{r}\right). \tag{5}$$

$$\hat{u} = \widehat{V_E} \cos\hat{\psi} + \widehat{V_N} \sin\hat{\psi}. \tag{6}$$

$$\hat{v} = -\widehat{V_E} \sin\hat{\psi} + \widehat{V_N} \cos\hat{\psi}. \tag{7}$$

- Between valid GNSS records, the estimate propagates position using held body velocities and propagates heading with IMU yaw rate.
  - The estimator reads only delivered records and timestamps supplied by the policy interface.
    - It does not read simulation truth, environmental forces, actual thrust, motor mismatch, or future packets. It does not subtract the configured gyro bias.
- Valid GNSS acquisition age must stay within 0.5 s and valid IMU acquisition age within 0.1 s.
  - Until both records exist, or after either becomes stale, the policy commands neutral PWM and clears its integrators.
    - This startup behavior explains the small initial speed dip in sensor-based runs.
- This observer is timestamp extrapolation rather than an extended Kalman filter.
  - It does not model acceleration during GNSS latency or estimate sensor bias.
    - The oracle case removes this shared estimation limitation and separates it from the controller law.

A.2 Common motor proxy and limits

State feedback uses estimates of total thrust and yaw moment as motor states. A common internal motor proxy follows the requested PWM after a nominal 35 ms delay and a 0.2 s first-order lag. The proxy uses the nominal signed M200 map and does not receive acknowledgments for missed commands.

$$\dot{\hat{F}}_i = \frac{F_{i,\mathrm{nom}} - \hat{F}_i}{\tau}, \quad \tau = 0.2 \ \ \mathrm{s}. \tag{8}$$

- Requested force is clipped between approximately -20.068 N and 40.207 N per thruster.
  - The default controller converts bounded force to PWM using the inverse installed M200 lookup.
    - This feedforward conversion is identical for all three controllers. Motor mismatch, lag, slew limits, dropout, and timing remain in the plant.
- Conditional anti-windup freezes all stored integrators when either force request clips.
  - Speed, cross-track, and PID heading integrals are also bounded by 2 m, 12 m s, and 2 rad s, respectively.
    - The current integration candidate contributes to the command, but is stored only when feedback is ready and neither thruster clips.
- `--linear-pwm` replaces the inverse lookup with the nominal trim tangent.
  - This is an independent compensation experiment, not a change in feedback gains.
    - Keep the simulator nonlinear-thrust flag enabled when evaluating compensation on the full boat model.

A.3 Linear design model

The full nonlinear plant supplies the local design model. Linearization occurs at $u = U$, $v = r = \psi = 0$, zero environmental forcing, and $F_p = F_s = 7$ N. Wind remains an external disturbance because the feedback gains should not require its true value. The underlying marine equations and coupling follow the simulator's Fossen-style model [1].

- Added inertia is included in the design.
  - Nominal loaded mass is 21.9 kg; effective surge and sway masses are 24.09 kg and 32.85 kg, and effective yaw inertia is approximately 5.048 kg m².
    - The longitudinal drag derivative is $d_u = d_1 + 2d_2U = 24$ N s/m. Lateral and yaw damping are $d_v = 35$ N s/m and $d_r = 12$ N m s.
- The equilibrium total force is $S_0 = d_1U + d_2U\ |U| = 14$ N.
  - Motor sum and moment dynamics have the same time constant τ.
    - Saturation, deadband, variable map slope, waves, noise, and timing errors are evaluated in simulation rather than inserted into the linear design matrices.

A.3.1 Surge subsystem

Define the augmented surge state $z_s = (\delta u, \delta S, I_u)^T$, where $\delta S = F_p + F_s - S_0$ and $\dot{I}_u = \delta u$. Its control input is $\delta S_c = S_c - S_0$.

$$\dot{z_s} = A_s z_s + B_s \delta S_c. \tag{9}$$

$$A_s = \begin{pmatrix} -\frac{d_u}{m_x} & \frac{1}{m_x} & 0 \\ 0 & -\frac{1}{\tau} & 0 \\ 1 & 0 & 0 \end{pmatrix}, \quad B_s = \begin{pmatrix} 0 \\ \frac{1}{\tau} \\ 0 \end{pmatrix}. \tag{10}$$

A.3.2 Sway-yaw subsystem

The lateral augmented state is $z_l = \left(e_y, \psi, v, r, M, I_y\right)^T$, where $M = \frac{b(F_s - F_p)}{2}$ and $\dot{I}_y = e_y$. The input is desired yaw moment M_c.

$$\dot{z_l} = A_l z_l + B_l M_c. \tag{11}$$

$$A_l = \begin{pmatrix} 0 & U & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & -\frac{d_v}{m_y} & -m_x\frac{U}{m_y} & 0 & 0 \\ 0 & 0 & (m_x - m_y)\frac{U}{I_z} & -\frac{d_r}{I_z} & \frac{1}{I_z} & 0 \\ 0 & 0 & 0 & 0 & -\frac{1}{\tau} & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}. \tag{12}$$

$$B_l = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \frac{1}{\tau} \\ 0 \end{pmatrix}. \tag{13}$$

- The surge and lateral controllability ranks are 3 and 6, matching their state counts.
  - The implemented matrix coefficients are checked against central finite differences of the full nonlinear hull.
    - Separating the local subsystems does not remove nonlinear surge/sway/yaw coupling from the simulator.

A.4 Three controller designs

The common model and interfaces allow three different feedback choices. PID organizes feedback into familiar loops. LQR chooses gains from a quadratic cost, while pole placement chooses the local response rates explicitly. Both state-feedback controllers use the same states and integral definitions, so their different results arise from their gain choices.

A.4.1 Cascaded PID

The proportional-integral-derivative (PID) controller regulates surge speed and heading. An outer cross-track proportional-integral loop converts line error into desired heading, and the measured sideslip estimate compensates the difference between hull heading and direction of travel.

$$\beta = \operatorname{atan2}\,(\hat{v}, \max(\hat{u}, 0.2)). \tag{14}$$

$$\psi_d = \operatorname{clip}\,\left(-k_y e_y - k_{Iy} I_y - \beta, -35°, 35°\right). \tag{15}$$

$$e_\psi = \operatorname{wrap}\,\left(\hat{\psi} - \psi_d\right). \tag{16}$$

$$S_c = S_0 - k_{Pu}\delta u - k_{Iu} I_u - k_{Du} q. \tag{17}$$

$$M_c = -k_{P\psi} e_\psi - k_{I\psi} I_\psi - k_{D\psi} \hat{r}. \tag{18}$$

- The speed derivative is filtered with time constant 0.1 s.
  - With tick interval h, the filter coefficient is $\alpha = \frac{h}{0.1+h}$.
    - Update $q_k = q_{k-1} + \alpha\left(\frac{\delta u_k - \delta u_{k-1}}{h} - q_{k-1}\right)$. Heading derivative action uses measured yaw rate and therefore avoids a derivative kick from a heading-reference step.
- The cross-track loop is slower than the heading loop.
  - Its heading clamp prevents large line errors from requesting a near-sideways course.
    - The design requires forward motion. The denominator floor regularizes the sideslip computation; it does not establish zero-speed controllability.
- PID tuning used an initial engineering choice and a heading-damping adjustment after an exploratory full-model run.
  - The final gains below were frozen for all cases and seeds.
    - No claim is made that these gains are globally optimal or that all three methods had equal optimization effort.

Loop	Gains	Interpretation
Surge PID	35, 12, 4	Force per speed, integrated speed, and speed derivative.
Heading PID	12, 1, 15	Moment per heading, integrated heading, and yaw rate.
Cross-track PI	0.18, 0.005	Heading per cross-track error and integrated cross-track error.

A.4.2 LQR with integral action

The linear quadratic regulator (LQR) minimizes an infinite-horizon quadratic cost for each augmented subsystem. The state integral therefore enters the cost along with the physical states. SciPy solves the continuous algebraic Riccati equation [2].

$$J = \int_0^\infty \left(z^T Q z + a^T R a\right) \mathrm{d}t. \tag{19}$$

$$A^T P + P A - P B R^{-1} B^T P + Q = 0. \tag{20}$$

$$K = R^{-1} B^T P, \quad a = -K z. \tag{21}$$

- The surge weights are $Q_s = \operatorname{diag}(25, 0.0025, 4)$ and $R_s = 0.01$.
  - The lateral weights are $Q_l = \operatorname{diag}(4, 25, 11.111111, 25, 0.04, 0.111111)$ and $R_l = 0.0625$.
    - These engineering weights normalize speed, line error, angular motion, motor effort, and integral accumulation; they do not come from a measured optimum.
- Commands satisfy $S_c = S_0 - K_s z_s$ and $M_c = -K_l z_l$.
  - The integral coordinates create constant force and heading biases needed to reject steady disturbances.
    - The resulting stability statement applies locally to the unconstrained continuous linear model. The implemented sampled nonlinear system is checked by simulation.

A.4.3 Pole placement with integral action

Pole placement uses the same augmented models but requests eigenvalues directly. SciPy calculates K so that the eigenvalues of $A - BK$ match the specified poles [3].

- Surge poles are $(-0.5, -1.3, -5.5)$ s⁻¹.
  - Lateral poles are $(-0.25, -0.4, -0.7, -1.2, -3.0, -5.5)$ s⁻¹.
    - The slower lateral modes allow line and integral recovery; faster modes govern yaw and motor response. Distinct poles avoid repeated-pole restrictions on each single-input subsystem.
- Commands use the same $S_c = S_0 - K_s z_s$ and $M_c = -K_l z_l$ form.
  - The test checks the achieved eigenvalues numerically.

- Pole placement controls local response rates but does not itself minimize effort or guarantee robustness to latency, saturation, or model error.

A.4.4 Numerical gains and local poles

The following gains are loaded directly from the saved design. Their units follow the state coordinates and force or moment output; gains across different coordinates should not be compared as dimensionless numbers.

A.4.4.1 Surge gains

Method	Gain row in the declared state order
LQR + integral	36.083, 0.36, 20
Pole placement	20.572, 0.261, 17.224

LQR closed-loop poles in s^{-1}. −5.211, −2.226, −0.358.
Pole placement closed-loop poles in s^{-1}. −5.5, −1.3, −0.5.

A.4.4.2 Lateral gains

Method	Gain row in the declared state order
LQR + integral	13.469, 44.028, −1.054, 18.748, 0.768, 1.333
Pole placement	11.855, 37.045, −1.752, 13.023, 0.521, 1.313

LQR closed-loop poles in s^{-1}. −5.056 + 1.036i, −5.056 - 1.036i, −0.186, −0.524, −0.73 + 0.085i, −0.73 - 0.085i.
Pole placement closed-loop poles in s^{-1}. −5.5, −3, −1.2, −0.25, −0.4, −0.7.

References

[1] T. I. Fossen, “Fossen's Marine Craft Model.” [Online]. Available: https://www.fossen.biz/html/marineCraftModel.html

[2] SciPy, “solve_continuous_are.” [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.linalg.solve_continuous_are.html

[3] SciPy, “place_poles.” [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.place_poles.html